

Introduction to Machine Learning for Security

Why security outgrew its rules — and how machines started to learn them

Prof. Dr.-Ing. Sebastian Schlesinger

HWR Berlin

The hook: security that no longer scales

Introduction to Machine Learning for Security

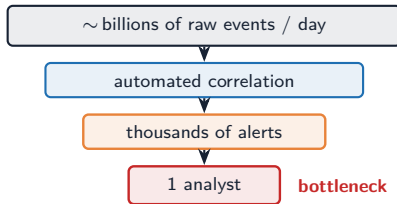
You are on the SOC night shift

- It is 03:00. You are the only analyst on shift.
- Your **Security Information and Event Management (SIEM)** shows the events collected in the last 24 h.
- Each one *could* be the first step of a breach — or nothing.

📌 Ask the room

How many security events does a mid-size enterprise SOC ingest per day? Shout a number.

(Order of magnitude: 10^9 – 10^{10} events/day. One human reviews maybe a few hundred.)



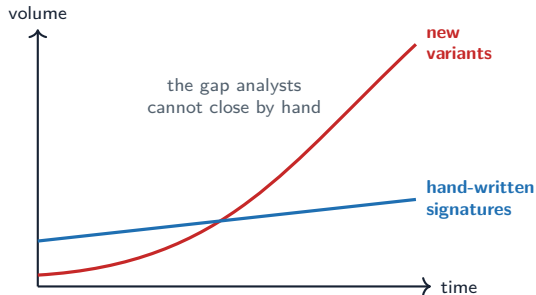
The funnel is real. The bottleneck is human.

The number that breaks the old model

- **AV-TEST** registers roughly **450,000 new malware samples every day** — > 100 per second.
- Classic antivirus works by **signatures**: a fingerprint per known bad file.
- So the honest question is very simple...

📖 Ask the room

Can we write — and ship — one hand-crafted signature for every new sample, forever?



Three everyday problems, one shared shape

Problem	Why a fixed rule struggles	What we actually want
Spam in your inbox	Wording changes hourly; “free” vs “fr_ee” vs an image	keep good mail, drop bad
Malware on an endpoint	450k new samples/day, packers, polymorphism	allow benign, block malicious
Zero-day intrusion	By definition <i>no</i> signature exists yet	flag “this is not normal”

The shape of all three

A stream of **objects** arrives; for each we must make a fast **good / bad** (or **normal / weird**) **decision**, at a volume and variability no analyst and no static list can keep up with.

Why classical methods hit a wall

Introduction to Machine Learning for Security

The classical toolbox — which you already master

Method	How it decides	You met it in
Rule-based systems	hand-written <code>if...then</code> logic	Firewalls, SIEM correlation
Heuristics	expert “rules of thumb”, scoring	Spam filters, EDR
Firewalls / ACLs	allow/deny by port, IP, protocol	Network Security
IDS/IPS (signature)	match traffic to known attack patterns	Intrusion Detection

Common thread: a *human* encodes “what bad looks like” *in advance*, explicitly.

A rule, made concrete — a YARA signature

```
rule Suspicious_Downloader {  
  meta:  
    author = "SOC team"  
  strings:  
    $a = "http://evil.example/payload"  
    $b = { 6A 40 68 00 30 00 00 } // opcodes  
    $c = "InternetOpenA"  
  condition:  
    2 of ($a, $b, $c)  
}
```

- Precise, fast, **explainable**: you know *exactly* why it fired.
- Zero false positives — *on what it was written for*.

Ask the room

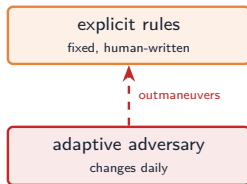
You are the attacker. Name *one* trivial change that defeats this rule.

What is YARA?

YARA is a tool designed to identify and classify malware. It uses a simple syntax to define rules that can be used to detect specific patterns in files or memory.

Where the wall is — five structural limits

1. **Scale.** One rule per threat $\times$ millions of new threats = a list nobody can maintain.
2. **Novelty.** A **zero-day** has, by definition, no signature. Rules only catch the past.
3. **Evasion.** Rules are public logic; **polymorphism** and obfuscation are cheap for the attacker.
4. **Combinatorics.** “Bad” is a fuzzy blend of hundreds of weak hints — too many to enumerate by hand.
5. **Maintenance.** Every rule is a false-positive risk and a forever cost. Rulesets rot.



We need decisions that *generalise* beyond the exact examples we wrote down.

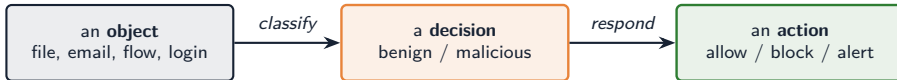
✗ Common misconception

“Then just write better/more rules.” — More rules means more false positives, more maintenance, and still zero coverage of the genuinely new.

Security is pattern recognition

Introduction to Machine Learning for Security

The re-frame: it was a classification problem all along



Classification

"Which known *category* does this object belong to?" — benign vs malicious, spam vs ham, this-of-5-malware-families.

Anomaly detection

"Does this object *deviate* from normal?" — for when we cannot enumerate "bad", only "usual" (great for zero-days).

Signatures were always doing classification — just with a rulebook a human had to write by hand.

Your whole curriculum, viewed as pattern recognition

Task	Object	Decision	Why explicit rules get hard
Malware detection	a file / process	benign / malicious	packers, polymorphism, 450k/day
Spam detection	an email	spam / ham	wording mutates hourly
Phishing detection	a URL / page	phishing / legit	look-alikes, new domains constantly
Intrusion detection	a network flow	normal / attack	encrypted, blends into traffic
UEBA	a user's behaviour	typical / suspicious	"normal" differs per person
Fraud detection	a transaction	legit / fraud	fraud mimics real behaviour
Cloud anomalies	API call / config	expected / risky	millions of calls, changing baselines
DNS tunnelling	DNS queries	benign / exfiltration	looks like ordinary DNS
Bot detection	a session / client	human / bot	bots imitate humans

Same shape every row: *object* → *decision*, where "bad" is too varied, too new, or too subtle to fully write down.

Two worked examples — object, decision, why rules fail

DNS tunnelling

Object: the stream of DNS queries from a host.

Decision: ordinary name lookups vs data smuggled out.

Why rules fail: a single query looks fine; the *pattern* (length, entropy, rate, rare subdomains) is the tell — and thresholds that catch it also flag legit CDNs.

UEBA (insider / account takeover)

Object: what one user does today.

Decision: their usual behaviour vs someone else on their account.

Why rules fail: there is no global “normal” — a 2 a.m. login is fine for an admin, alarming for HR. Normal is *per person*.

The uncomfortable conclusion

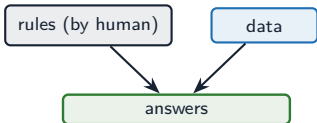
For these, nobody can hand-write the rule — but we *do* have piles of **examples** of normal and abnormal.
What if the machine could read the rule off the examples itself?

Machine learning, intuitively

Introduction to Machine Learning for Security

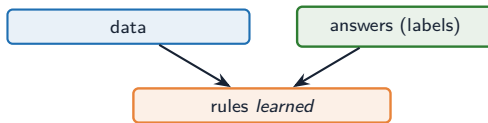
The one idea behind all of machine learning

Classical



humans write rules → program applies them

Machine learning



show examples + answers → machine infers the rule

In one line

Machine learning = instead of *writing* the decision rule, we *learn* it from examples — so it can generalise to objects we never explicitly listed.

🔗 Bridge to classical security

A signature is a rule you wrote. A trained model is a rule the data wrote. Same job (classify), different author.

Data, features, labels — said without maths

- **Data:** the raw objects — e.g. 100k emails you kept.
- **Features:** the measurable clues per object. For an email: *#links, % capitals, has “invoice”, sender age, attachment type...* — turning an object into numbers.
- **Label:** the known answer for training examples — spam or ham.
- Feature engineering *is* the security expertise: you already know which clues matter.

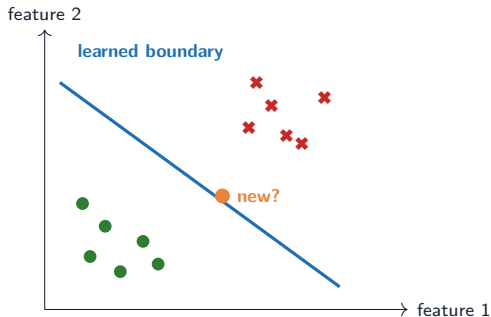
email	#links	%CAPS	“invoice”	label
e ₁	9	40	yes	spam
e ₂	1	5	no	ham
e ₃	12	60	yes	spam
e ₄	0	8	no	ham
e ₅	7	33	yes	spam

Each row = one object as a *feature vector* + its label.

🔗 Bridge to classical security

“Features” are exactly the *indicators* you put in a YARA or SIEM rule — but now they are *inputs to be weighed*, not a hard threshold you fix by hand.

Learning a function = drawing the boundary

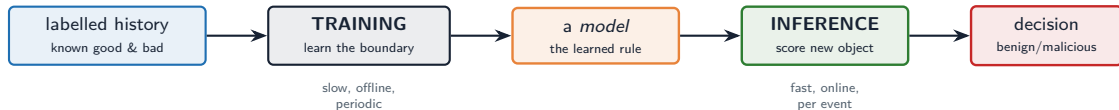


- **Training:** nudge the boundary until it separates the labelled examples well.
- **Inference:** a new object arrives → which side of the boundary is it on? → that is the prediction.
- **Generalisation:** the boundary must work on *unseen* objects (the orange point), not just memorise the training set.

✗ Common misconception

“The model memorises all the malware.” It does not store the samples — it learns a *boundary*. That is why it can flag a variant it has never seen.

Training vs. inference — the two clocks of every ML system



🔗 Bridge to classical security

Inference is the run-time check — the moral equivalent of the IDS matching a packet. It just uses a *learned* rule instead of a written one.

✗ Common misconception

“The AI keeps learning live from every attack.” Usually not: most systems *train periodically* and only *infer* in real time. Continuous learning is a deliberate (and risky) design choice.

The three learning paradigms

Introduction to Machine Learning for Security

Three ways a machine can learn — each maps to security

Supervised

learn from *labelled* examples
(good/bad known)

Unsupervised

find structure in *unlabelled* data
(what is “normal”?)

Reinforcement

learn by *trial & reward*
(act, get feedback)

Paradigm	You give it. . .	Security example
Supervised	objects <i>plus</i> the right answers	spam/malware/phishing classifiers
Unsupervised	objects only, no answers	UEBA, network anomaly detection
Reinforcement	an environment & a reward signal	auto-response, adaptive honeypots, fuzzing

Supervised learning — the workhorse of detection

- You have **labels**: a corpus of known malware *and* known benign files.
- The model learns the boundary between the two classes.
- New file → “**92% malicious**” → block.
- Most commercial malware/phishing/spam detection is here.

👉 Bridge to classical security

This is your AV/IDS *signature*, generalised: instead of matching one known-bad, it recognises the *family resemblance* to thousands of known-bad — so a fresh variant still trips it.

The catch (Part III)

Supervised learning is only as good as its labels. In security, labels are *scarce, expensive, and often wrong* — attackers hide, so “benign” may just be “not caught yet”.

✗ Common misconception

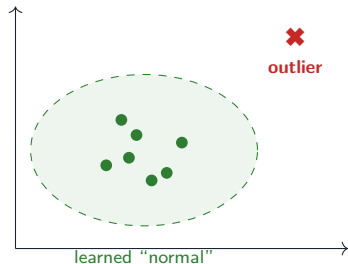
“More data always means a better model.”
Mislabelled or biased data makes it confidently wrong.

Unsupervised learning — when you cannot label “bad”

- No labels: you only have *normal* operations logged.
- The model learns a **baseline** of “usual”, then flags what *deviates* — **anomaly detection**.
- Perfect for **zero-days** and **insiders**: you need not have seen the attack before, only “this isn’t normal”.
- Also **clustering**: group similar malware into families automatically.

🔗 Bridge to classical security

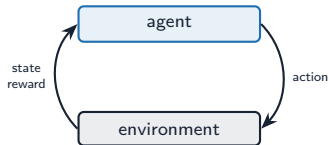
This is UEBA and network baselining — the ML version of “I know what my network usually looks like, and *that* is new”.



No labels needed — only a notion of “far from usual”.

Reinforcement learning — learning to act, not just to judge

- An **agent** takes an **action**, sees the result, gets a **reward**, and adjusts — learning a *policy* by trial.
- Security uses (mostly emerging):
 - **Automated response**: which containment action minimises damage?
 - **Adaptive honeypots** that keep an attacker engaged.
 - **Fuzzing / pentest agents** that learn which inputs break a target.
- The same idea powers game-playing agents (Go, Atari).



Least mature in production security — but the research frontier for autonomous defence *and* attack.

Bridge to classical security

Think of a **SOAR** playbook that *tunes itself*: it tries responses and keeps what worked — instead of a fixed, human-authored runbook.

Where ML lives in security today

Introduction to Machine Learning for Security

You are already surrounded by security ML

Where	What the ML does	Mostly which paradigm
Endpoint Detection and Response (EDR)	flag malicious processes/behaviour on endpoints	supervised + anomaly
Extended Detection and Response (XDR)	correlate signals across endpoint/network/cloud	supervised + unsup.
SIEM	prioritise & reduce alerts, spot outliers	anomaly detection
UEBA	per-user/-entity behavioural baselines	unsupervised
Malware det.	classify files/traffic as benign/malicious	supervised
Phishing det.	score URLs, pages, emails	supervised
Fraud det.	score transactions in real time	supervised + anomaly
Cloud sec.	anomalous API/IAM/config behaviour	anomaly detection

The new arrivals — foundation models & LLMs

- **Foundation models / LLMs** are supervised learning at huge scale — trained on text/code, then adapted.
- In defence (emerging, useful, not magic):
 - summarise alerts, draft incident reports, explain a suspicious script in plain language;
 - “**SOC copilot**”: translate an analyst question into a query, triage faster.
- In offence (why we must understand them):
 - scaled, fluent **phishing** and social engineering;
 - malware/exploit assistance, **prompt injection** of AI tools.

Same story, new scale

An LLM is still “learn a function from examples” — just with billions of parameters. The *principles* from today still apply; so do the *pitfalls* (next section) — amplified.

✗ Common misconception

“AI will replace the SOC analyst.” Today it *augments* — it drafts, ranks and explains; a human still decides and is accountable.

What makes ML in security special

Introduction to Machine Learning for Security

Why security ML $\neq$ “ML with a security dataset”

The defining difference

In most ML, the world is indifferent: cats do not restyle themselves to fool your classifier. In security, the data is generated by an **adaptive, intelligent adversary** whose *explicit goal* is to defeat your model.



Consequences we will unpack:

- the target moves *on purpose* $\Rightarrow$ models decay;
- errors are not symmetric $\Rightarrow$ FP vs FN cost differs wildly;
- you must *explain* a block, not just make it.

The six hazards every security ML system faces

1. **Adaptive adversary.** Attackers probe and adapt; a great model ages the moment it ships.
2. **Data / concept drift.** “Normal” and “malicious” both change — retrain or rot.
3. **Class imbalance & base rate.** 1 attack in a million events: even 99.9% accuracy drowns you in false positives.
4. **False positives / negatives.** A FP wastes analysts; a FN is a breach. The threshold is a business decision.
5. **Explainability.** “The model said so” will not survive an audit, a court, or an angry blocked user.
6. **Adversarial ML.** Attacks *on* the model itself — next slide.

Definitions

Recall: from all the malicious events, what fraction did we catch? It is defined as $\text{recall} = \frac{TP}{TP + FN}$.

Precision: from all the events we flagged, what fraction were actually malicious? It is defined as $\text{precision} = \frac{TP}{TP + FP}$.

False positive rate: from all the benign events, what fraction did we flag? It is defined as $\text{FPR} = \frac{FP}{FP + TN}$.

✗ Common misconception

“99% accuracy = great detector.” With a 1-in-a-million base rate, accuracy is meaningless — ask about false-positive *rate* and recall.

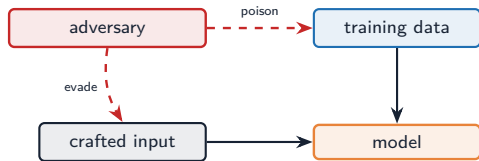
Adversarial machine learning — the model is now a target

Evasion (at inference time)

Perturb the input just enough to flip the decision: add benign strings to malware, tweak pixels in a CAPTCHA, pad a packet. The file still works — the model now says “benign”.

Poisoning (at training time)

Feed the training pipeline bad data so the model learns the *wrong* boundary — e.g. teach a spam filter that the attacker’s words are “ham”.



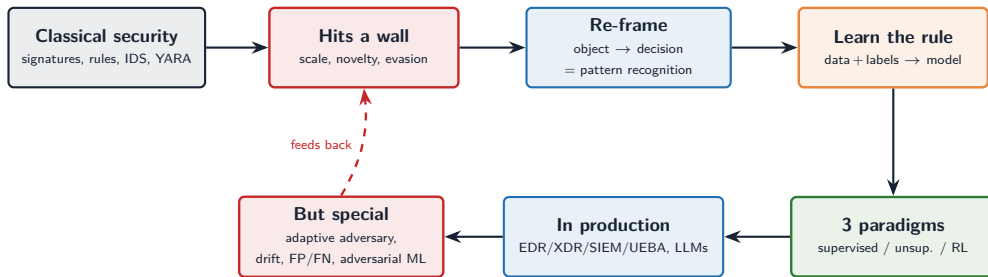
Two entry points: corrupt what it *learns*, or fool what it *sees*.

These — plus model theft, membership inference, and prompt injection — are dedicated lectures later in this series.

The red thread

Introduction to Machine Learning for Security

The board: one picture of the whole lecture



Follow the arrows (a snake): the problem forced the re-frame, the re-frame invited learning, and learning brought new problems — which loop back and drive the rest of the series.

Summary

So, is everything solved with ML?

- Remember, the idea to introduce ML was the *scale, novelty and evasion* limits of classical security.
- ML is more adaptive, but new problems arise: Data Poisoning, Evasion, Concept Drift, Explainability, Base Rate Fallacy
- What do we do then? We introduce new security mechanisms: adversarial training, certified robustness, secure datasets, explainability, uncertainty estimation
- So, a typical pattern in computer science: a shift of paradigm solves some problems, but introduces new ones. The cycle continues.

A recurring pattern in Computer Science

Machine Learning did not end the security arms race. It moved it. We no longer fight over rules—we fight over models.

Self-test — three quick questions

Q1. A **zero-day** attack evades signature-based detection mainly because:

- A) it is encrypted
- B) no signature for it exists yet
- C) it is too small
- D) firewalls are disabled

Q2. You must detect *unknown* attacks and have *no* labelled attack data. Best fit:

- A) supervised classification
- B) unsupervised anomaly detection
- C) writing more YARA rules
- D) reinforcement learning

Q3. With 1 attack per 1,000,000 events, why is “99.9% accuracy” a bad headline?

- A) accuracy can't be measured
- B) the base rate makes false positives dominate
- C) 99.9% is impossible
- D) anomalies don't exist

Answers on the next slide — decide before you turn the page.

Self-test — answer key

Q	Answer
---	--------

- | | |
|----|--|
| Q1 | B — a signature encodes the <i>past</i> ; a zero-day is new, so nothing matches. |
| Q2 | B — no labels $\Rightarrow$ model “normal” and flag deviations (anomaly detection). |
| Q3 | B — at a 1-in-a-million base rate, even 0.1% error = 1,000 false alarms per million. |
-

If Q3 stung

That is the **base-rate fallacy** — the single most important trap in security ML, and why analysts drown in alerts. We return to it in the detection-engineering lecture.

Key takeaways

- Classical security **did not fail** — it hit *scale, novelty and evasion* limits that hand-written rules structurally cannot cross.
- Most security tasks are really **classification** or **anomaly detection**: *object* → *decision*.
- **ML** = *learn* the decision rule from **data**, **features** and **labels** instead of writing it by hand.
- Three paradigms, three jobs: **supervised** (labelled detection), **unsupervised** (find normal / anomalies), **reinforcement** (learn to act).
- Security ML is unique because the **adversary adapts** — bringing drift, brutal base rates, explainability needs and **adversarial ML**.

You can now answer the opening question: *why did ML become necessary in security at all?*

The one message

Machine learning does **not replace** classical security —
it **extends** it everywhere explicit rules are no longer enough.

Signatures for the known. Learning for the new, the vast, and the changing. Together.

References

Introduction to Machine Learning for Security

References & further reading

- Sommer & Paxson — *Outside the Closed World: On Using Machine Learning for Network Intrusion Detection*. IEEE S&P, 2010. (The classic “why security ML is hard” paper.)
- Chio & Freeman — *Machine Learning & Security*. O'Reilly, 2018.
- Goodfellow, Bengio & Courville — *Deep Learning*. MIT Press, 2016 (intuition chapters).
- Biggio & Roli — *Wild Patterns: Ten Years After the Rise of Adversarial Machine Learning*. Pattern Recognition, 2018.
- NIST AI 100-2 — *Adversarial Machine Learning: Taxonomy and Terminology*. NIST, current edition.
- MITRE ATLAS — Adversarial Threat Landscape for AI Systems (atlas.mitre.org).
- OWASP *Top 10 for LLM Applications* — prompt injection & friends.
- AV-TEST malware statistics (av-test.org/en/statistics/malware).
- Apruzzese et al. — *The Role of Machine Learning in Cybersecurity*. ACM DTRAP, 2023.

Thank you

Questions, counter-examples, and “but what about. . .” welcome.

Prof. Dr.-Ing. Sebastian Schlesinger ■ HWR Berlin ■ Introduction to ML for Security